Spring Boot Interview Questions

1. What are the features of Spring Boot?

- Spring Boot CLI – This allows you to Groovy / Maven for writing Spring boot application and avoids boilerplate code.
- Starter Dependency – With the help of this feature, Spring Boot aggregates common dependencies together and eventually improves productivity and reduces the burden on
- Spring Initializer – This is a web application that helps a developer in creating an internal project structure. The developer does not have to manually set up the structure of the project while making use of this feature.
- Auto-Configuration – This helps in loading the default configurations according to the project you are working on. In this way, unnecessary WAR files can be avoided.
- Spring Actuator – Spring boot uses actuator to provide “Management EndPoints” which helps the developer in going through the Application Internals, Metrics etc.
- Logging and Security – This ensures that all the applications made using Spring Boot are properly secured without any hassle.

2. Differentiate between Spring and Spring Boot.

- The Spring Framework provides multiple features like dependency injection, data binding, aspect-oriented programming (AOP), data access, and many more that help easier development of web applications whereas Spring Boot helps in easier usage of the Spring Framework by simplifying or managing various loosely coupled blocks of Spring which are tedious and have a potential of becoming messy.
- Spring boot simplifies commonly used spring dependencies and runs applications straight from a command line. It also doesn’t require an application container and it helps in monitoring several components and configures them externally.

3. Explain the advantages of using Spring Boot for application development.

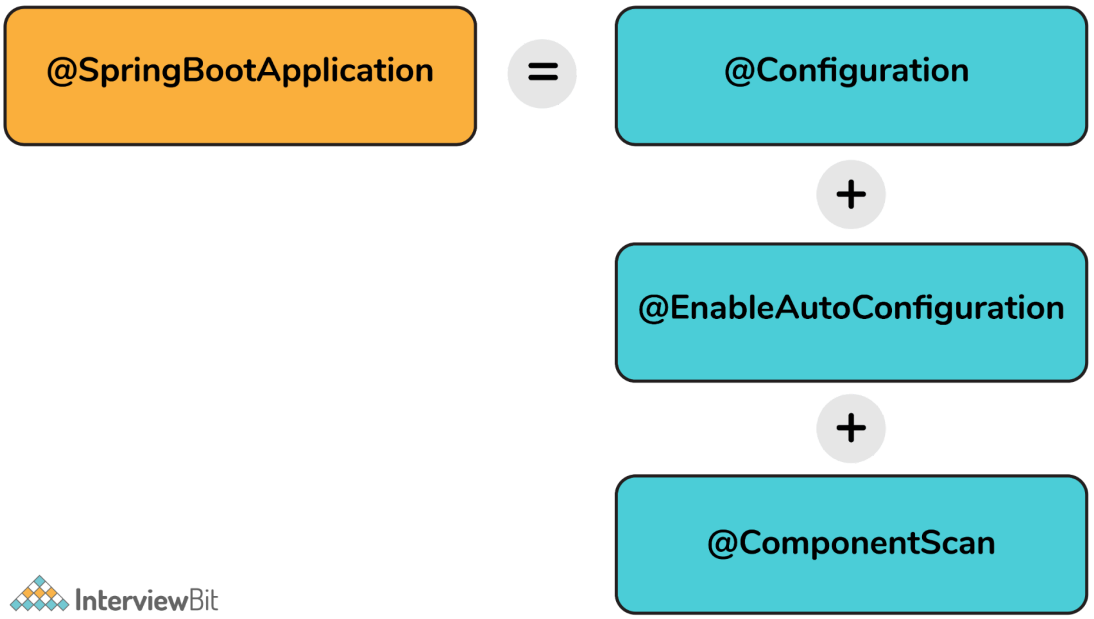
- Spring Boot helps to create stand-alone applications which can be started using java.jar (Doesn't require configuring WAR files).
- Spring Boot also offers pinpointed 'started' POMs to Maven configuration.
- Has provision to embed Undertow, Tomcat, Jetty, or other web servers directly.
- Auto-Configuration: Provides a way to automatically configure an application based on the dependencies present on the classpath.
- Spring Boot was developed with the intention of lessening the lines of code.
- It offers production-ready support like monitoring and apps developed using spring boot are easier to launch.

4. What do you understand by the term 'Spring Boot'?

Spring Boot is an open-source, java-based framework that provides support for Rapid Application Development and gives a platform for developing stand-alone and production-ready spring applications with a need for very few configurations.

5. What does @SpringBootApplication annotation do internally?

As per the Spring Boot documentation, the @SpringBootApplication annotation is one point replacement for using @Configuration, @EnableAutoConfiguration and @ComponentScan annotations alongside their default attributes.



This enables the developer to use a single annotation instead of using multiple annotations thus lessening the lines of code. However, Spring provides loosely coupled features which is why we can use these annotations as per our project needs.

6. What are the uses of @RequestMapping and @RestController annotations in Spring Boot?

- @RequestMapping:
  - This provides the routing information and informs Spring that any HTTP request matching the URL must be mapped to the respective method.
  - org.springframework.web.bind.annotation.RequestMapping has to be imported to use this annotation.
- @RestController:
  - This is applied to a class to mark it as a request handler thereby creating RESTful web services using Spring MVC. This annotation adds the @ResponseBody and @Controller annotation to the class.
  - org.springframework.web.bind.annotation.RestController has to be imported to use this annotation.

Check out more Interview Questions on Spring Boot here.

7. Can the default web server in the Spring Boot application be disabled?

Yes! application.properties is used to configure the web application type, by mentioning spring.main.web-application-type=none.

8. How to disable specific auto-configuration class?

- You can use the exclude attribute of @EnableAutoConfiguration for this purpose as shown below:

```
@EnableAutoConfiguration(exclude = {InterviewBitAutoConfiguration.class})
```

If the class is not specified on the classpath, we can specify the fully qualified name as the value for the excludeName.

```
//By using "excludeName"
@EnableAutoConfiguration(excludeName={Foo.class})
```

- You can add into the application.properties and multiple classes can be added by keeping it comma-separated.

9. Can you tell how to exclude any package without using the basePackages filter?

We can use the exclude attribute while using the annotation @SpringBootApplication as follows:

```
@SpringBootApplication(exclude= {Student.class})
public class InterviewBitAppConfiguration {}
```

10. Can we change the default port of the embedded Tomcat server in Spring boot?

- Yes, we can change it by using the application properties file by adding a property of server.port and assigning it to any port you wish to.
- For example, if you want the port to be 8081, then you have to mention server.port=8081. Once the port number is mentioned, the application properties file will be automatically loaded by Spring Boot and the specified configurations will be applied to the application.

11. What are the possible sources of external configuration?

- Spring Boot allows the developers to run the same application in different environments by making use of its feature of external configuration. This uses environment variables, properties files, command-line arguments, YAML files, and system properties to mention the required configuration properties for its corresponding environments. Following are the sources of external configuration:
    - Command-line properties – Spring Boot provides support for command-line arguments and converts these arguments to properties and then adds them to the set of environment properties.
    - Application Properties – By default, Spring Boot searches for the application properties file or its YAML file in the current directory of the application, classpath root, or config directory to load the properties.
    - Profile-specific properties – Properties are loaded from the application-{profile}.properties file or its YAML file. This file resides in the same location as that of the non-specific property files and the {profile} placeholder refers to an active profile or an environment.

12. What is Spring Boot dependency management system?

- It is basically used to manage dependencies and configuration automatically without the need of specifying the version for any of that dependencies.

13. What are the effects of running Spring Boot Application as "Java Application"?

- The application automatically launches the tomcat server as soon as it sees that we are running a web application.